

RTCC_18FxxJ50 library for mikroPascal PRO for PIC

Author: VCC

Version: 1

2011

Features:

- Get/Set RTCC/Alarm
- Get RTCC/Alarm/ElapsedTimeToAlarm as string
- BCD and Dec versions of Get / Set routines for RTCC/Alarm/ElapsedTimeToAlarm
- Alarms can be masked to trigger at different intervals
- Designed for use with DeepSleep
- Backup library configuration state to DeepSleep persistent registers
- Compiler directive for synchronizing RTCC/Alarm register reads with RTCSYNC bit
- Compiler directive for synchronizing RTCC/Alarm register writes with RTCSYNC bit
- Compiler directive for updating RTCC DateTime registers in Read-Modify-Write fashion
- Compiler directive for double sampling RTCC DateTime registers on reading
- Compiler directive for saving library configuration state to DeepSleep persistent registers

Requirements / Recommendations:

- If using DeepSleep, *RTCCConfiguredAfterPowerUp* variable must be reset to 0 on PIC init.
- If using DeepSleep, *RTCCConfiguredAfterPowerUp* variable may be saved to DeepSleep persistent registers to preserve the information about RTCC "configured state" to avoid reconfiguring after wake-up.

Limitations:

- *GetRTCCElapsedToNextAlarm* BCD/Dec routines do not take into account the alarm mask

Table of contents:

1. Compiler directives defined by the library
2. Routines
 - 2.1. Control routines
 - 2.2. BCD routines
 - 2.3. Dec routines
 - 2.4. RTCC/alarm string routines
 - 2.5. RTCC state backup routines
3. Global constants
 - 3.1. EnableRTCCModule constants
 - 3.2. Alarm Options constants
 - 3.3. Day of week constants
 - 3.4. DeepSleep GP Persistent selector constants
 - 3.5. GetRTCCDateTimeStr constants
 - 3.6. DateTime format constants
 - 3.7. Config backup mask constant
4. Global variables
5. Usage example

1. Compiler directives defined by the library

The RTCC_18FxxJ50 library features five compiler directives with the purpose of including code that may improve the operation with the expense of used ROM.

Three of the four compiler directives, *UseRTCSYNCReads*, *UseRTCSYNCWrites* and *SampleTwiceOnDateTimeReading*, directly affect the synchronization requirements of the RTCC module when reading or writing from/to its registers. One of the other compiler directives, *UseReadModifyWriteDateTime*, has the purpose of adding code that allows updating all the Date/Time registers, when setting either Date or Time, while the last one, *HasDeepSleep* includes two routines for saving the RTCC "configured state" to one of the available deep sleep persistent registers (if available).

UseRTCSYNCReads compiler directive adds code that makes the CPU wait for **RTCSYNC** bit when reading either the date or time. The added code is a blocking code and thus, any "Get RTCC Date/Time" or "Get Alarm DateTime" routine must be called only after enabling RTCC module. Also, when using in combination with *UseReadModifyWriteDateTime*, the RTCC must be enabled prior to using *SetRTCCDateBCD* and *SetRTCCDateDec* routines, because these routines call the complementary reading routines, which are affected by *UseRTCSYNCReads* compiler directive.

UseRTCSYNCWrites compiler directive adds code that makes the CPU wait for **RTCSYNC** bit when writing either the Date or Time. The added code is a blocking code and thus, any "Set RTCC Date/Time", "Set Alarm DateTime" or the *SetRTCCAlarmRepeat* routine must be called only after enabling RTCC module.

The synchronization routine, linked when either *UseRTCSYNCReads* or *UseRTCSYNCWrites* is defined, waits for logic "1" to logic "0" transition of the **RTCSYNC** bit. This routine is a blocking call to properly detect the beginning of the "0" logic state of the **RTCSYNC** bit. According to PIC18F46J50 datasheet, synchronizing reads or writes with this bit are not mandatory, but are recommended.

SampleTwiceOnDateTimeReading compiler directive adds code that reads the Date or Time registers two times in order to provide consistent information. Sampling the Date or Time registers, two times, is also a datasheet recommendation, thus including this compiler directive is left up to the user.

UseReadModifyWriteDateTime compiler directive makes the setting of date or time in a read-modify-write fashion. This means that, when setting the date, both date and time are read, the date variables are updated and then, both date and time are written. When setting the time, both date and time are read, the time variables are updated and then, both date and time are written. This option allows the possibility to avoid the inconsistent setting of date or time when one or more registers of the RTCC module are near a rollover.

In order for this option to have effect, one of the *UseRTCSYNC* compiler directives must be included. Although it is unusual to set date or time near such a rollover, this option wants to simplify the firmware to avoid checking for date/time approaching a rollover. For example, setting date when time is 23:59:59, then setting time, may result in an incremented date, i.e. a date after the set one. By setting both date and time as a whole (with *UseReadModifyWriteDateTime* included), the RTCC date/time registers are updated as a single sample. The routines affected by this option are: *SetRTCCTimeBCD*, *SetRTCCDateBCD*, *SetRTCCTimeDec* and *SetRTCCDateDec*.

HasDeepSleep compiler directive should be included only if the used microcontroller has deep sleep persistent registers. Otherwise, the library won't compile.

The user should test all combinations of these compiler directives, because they may also depend on application. The library works with all directives undefined, but it is recommended that *UseRTCSYNCWrites* and *SampleTwiceOnDateTimeReading* be included.

2. Routines

To benefit the features of the RTCC module, the library makes available routines for configuring the RTCC and the alarm, for setting and reading the date/time registers, reading date/time as string and saving the library "configured state" to one of the two DeepSleep Persistent registers.

2.1. Control Routines

procedure `EnableRTCCModule`(Options: Byte);

This routine enables the PIC internal RTCC module and the T1 oscillator according to *Options* parameter. If the T1 oscillator is used as the RTCC clock source, this oscillator must be up and running before any call to Get/Set routines when at least one of the `UseRTCSYNC` compiler directives is included. The *Options* parameter can be set with several constants provided by the library, or combinations of these constants. See *EnableRTCCModule constants* in section 3 of this documentation for further details of how to call the `EnableRTCCModule` routine.

procedure `DisableRTCCModule`;

Disables the RTCC module and disables also the T1 oscillator if it was enabled with the *Options* parameter of `EnableRTCCModule` routine.

procedure `SetRTCCCalibrationValue`(Calib: Short);

This procedure sets the calibration register of the RTCC module. If T1 oscillator is running when calling this routine, it is disabled prior to writing the **RTCCAL** register, then reenabled (if was enabled before). The content of *Calib* parameter is directly written to **RTCCAL** register. Setting a positive value to *Calib* parameter makes the RTCC run faster and setting a negative value, makes the RTCC run slower.

procedure `EnableRTCCAlarm`(Options: Byte);

Enables the alarm and sets its trigger options. For setting the Options parameter, check the possible constants from the *Alarm Options constants* section. The alarm disables itself after the last trigger.

procedure `DisableRTCCAlarm`;

Disables the alarm. If the alarm is continuously triggering (e.g. with chime option), it is stopped.

procedure `SetRTCCAlarmRepeat`(NumberOfTimes: Byte);

This routine configures the number of times an alarm is triggered, once the alarm counters match the RTCC counters.

2.2. BCD Routines

procedure `GetRTCCDateTimeBCD`(var Year, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

This routine reads the date and time in BCD format (as read from the RTCC registers).

procedure `SetRTCCDateTimeBCD`(Year, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

This routine sets the date and time in BCD format. The *Year* parameter is two a digit value, for example, 11 for 2011. For *WeekDay* parameter, see *Day of week constants*.

procedure `GetRTCCTimeBCD`(var WeekDay, Hours, Minutes, Seconds: Byte);

Reads the RTCC time, including weekday in BCD format.

procedure `SetRTCCTimeBCD`(WeekDay, Hours, Minutes, Seconds: Byte);

Sets the RTCC time in BCD format. For *WeekDay* parameter, see *Day of week constants*.

procedure **GetRTCCDateBCD**(**var** Year, Month, Date: Byte);

Reads the RTCC date, in BCD format.

procedure **SetRTCCDateBCD**(Year, Month, Date: Byte);

Sets the RTCC date in BCD format.

procedure **GetRTCCAlarmBCD**(**var** Mask, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

Reads the alarm date and time in BCD format and alarm trigger mask in decimal format.

procedure **SetRTCCAlarmBCD**(Mask, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

Sets the alarm date and time in BCD format and alarm trigger mask in decimal format. For *WeekDay* parameter, see *Day of week constants*.

procedure **GetRTCCElapsedToNextAlarmBCD**(**var** Days: Word; **var** Hours, Minutes, Seconds: Byte);

Reads, in BCD format, the elapsed number of days, hours, minutes and seconds until the alarm triggers if configured with *CArm_Mask_OnceAYear* mask. The computation of the elapsed time is limited to this alarm mask. Setting the alarm with other masks will normally result in triggering earlier than computed by this routine. However, this limitation does not affect the operation of the RTCC or alarm.

2.3. Dec Routines

procedure **GetRTCCDateTimeDec**(**var** Year, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

This routine reads the RTCC date and time in decimal format.

procedure **SetRTCCDateTimeDec**(Year, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

This routine sets the RTCC date and time in decimal format. For year 2011, set *Year* parameter to 11. For *WeekDay* parameter, see *Day of week constants*.

procedure **GetRTCCTimeDec**(**var** WeekDay, Hours, Minutes, Seconds: Byte);

Reads the RTCC time in decimal format.

procedure **SetRTCCTimeDec**(WeekDay, Hours, Minutes, Seconds: Byte);

Sets the RTCC time in decimal format. For *WeekDay* parameter, see *Day of week constants*.

procedure **GetRTCCDateDec**(**var** Year, Month, Date: Byte);

Reads the RTCC date in decimal format.

procedure **SetRTCCDateDec**(Year, Month, Date: Byte);

Sets the RTCC date in decimal format. For year 2011, set *Year* parameter to 11.

procedure **GetRTCCAlarmDec**(**var** Mask, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

Reads the alarm date, time and mask in decimal format.

procedure **SetRTCCAlarmDec**(Mask, Month, Date, WeekDay, Hours, Minutes, Seconds: Byte);

Sets the alarm date, time and mask in decimal format. For *Mask* parameter, see *Alarm Options constants*. For *WeekDay* parameter, see *Day of week constants*.

procedure **GetRTCCElapsedToNextAlarmDec**(**var** Days: Word; **var** Hours, Minutes, Seconds: Byte);

Reads, in decimal format, the elapsed number of days, hours, minutes and seconds until the alarm triggers if configured with *CArm_Mask_OnceAYear* mask. The computation of the elapsed time is limited to this alarm mask. Setting the alarm with other masks will normally result in triggering earlier than computed by this routine. However, this limitation does not affect the operation of RTCC or alarm.

All BCD and Dec routines that get or set the year use the two digit format for the *Year* parameter. For example, for year 2011, the *Year* parameter is 11. The hour parameter uses the 24h format.

2.4. RTCC/alarm string routines

function `GetRTCCDateTimeStr`(WhatToDisplay: Byte): string[30];

Returns a string containing either the current DateTime, or the current alarm, or the elapsed time to next alarm. These three options are set through the *WhatToDisplay* parameter. For possible values of this parameter, see *GetRTCCDateTimeStr constants*.

For time, it can display, e.g. *Wednesday, 2011.08.10 22:51:00*.

For alarm, it can display, e.g. *Friday, 2011.08.12 23:27:20*.

For time elapsed to next alarm, it can display, e.g. *2d 0h 36m 0s*. (meaning 2 days, 0 hours, 36 minutes, and 0 seconds).

procedure `SetRTCCDateStrFormat`(DateStrFormat: Byte);

Sets the year / month / date format used by *GetRTCCDateTimeStr* routine when displaying either current RTCC time or current alarm. For possible values of the *DateStrFormat* parameter, see *DateTime format constants*.

2.5. RTCC state backup routines

procedure `SaveRTCCConfigStateToDeepSleepPersistentRegs`(DSGPRESel: Byte);

Saves the library configuration options to one of the two DeepSleep persistent registers. This procedure assumes that the used device has such registers. It should be called before preparing to enter deep sleep mode. This routine has the purpose of allowing the user to avoid reconfiguring the RTCC after waking from deep sleep. The *DSGPRESel* parameter selects one of the two DeepSleep persistent registers. For possible values of *DSGPRESel* parameter, see *DeepSleep GP Persistent selector constants*.

procedure `RestoreRTCCConfigStateFromDeepSleepPersistentRegs`(DSGPRESel: Byte);

Updates the library configuration options with those saved before entering deep sleep. This routine allows avoiding reconfiguring the RTCC after waking from deep sleep. The *DSGPRESel* parameter selects one of the two DeepSleep persistent registers. For possible values of *DSGPRESel* parameter, see *DeepSleep GP Persistent selector constants*.

3. Global constants

This library makes available several constants for making easy the use of its routines. Most of these constants can be "ORed" together for obtaining several options. Nevertheless, some of them should not be used like this.

3.1. EnableRTCCModule constants

The *EnableRTCCModule* constants are used by [EnableRTCCModule](#) routine in its *Options* parameter. Every constant from this group corresponds to one or two bits in the *Options* parameter. Most of these constants can be "ORed" together.

```
const
CRTCC_StartT1Osc: Byte = 128;
CRTCC_DisableT1OscOnDisableRTCC: Byte = 64;
CRTCC_RTCCOutputEnable: Byte = 32;
CRTCC_DisIntOnWrite: Byte = 16;
CRTCC_DisIntOnRead: Byte = 8;
CRTCC_DisIntOnReadWrite: Byte = CRTCC_DisIntOnWrite or
CRTCC_DisIntOnRead;
CRTCC_RTCCPinOutputOsc: Byte = 4;
CRTCC_RTCCPinOutputSeconds: Byte = 2;
CRTCC_RTCCPinOutputAlarm: Byte = 0;
```

CRTCC_StartT1Osc – if set, the [EnableRTCCModule](#) routine starts the T1 oscillator.

CRTCC_DisableT1OscOnDisableRTCC – if set, stops the T1 oscillator on [DisableRTCCModule](#).

CRTCC_RTCCOutputEnable – if set, makes the RTCC pin available to the RTCC module.

CRTCC_DisIntOnWrite – if set, disables interrupts when writing to RTCC or alarm registers.

CRTCC_DisIntOnRead – if set, disables interrupts when reading from RTCC or alarm registers.

CRTCC_DisIntOnReadWrite – if set, disables interrupts when reading from and writing to RTCC or alarm registers.

CRTCC_RTCCPinOutputOsc – if set, makes the RTCC pin output the RTCC clock signal.

CRTCC_RTCCPinOutputSeconds – if set, makes the RTCC pin output the RTCC seconds signal. This option should not be "ORed" with CRTCC_RTCCPinOutputOsc.

CRTCC_RTCCPinOutputAlarm – is the default value if none of the above "RTCCPinOutput" constants are included. In this case, the RTCC pin outputs the alarm pulse with a frequency divided by two.

3.2. Alarm Options constants

These constants set the *chime* feature and the *mask* feature of the alarm module. The *chime* option can be "ORed" with any of the *mask* features, but only one *mask* option may be used at a time. These constants are used by the [EnableRTCCAlarm](#) routine and directly written to **ALRMCFG** register.

```
const
CAlarm_EnableChime = 64;

CAlarm_Mask_EveryHalfSecond: Byte = 00000000;
CAlarm_Mask_EverySecond: Byte = 00000100;
CAlarm_Mask_Every10Seconds: Byte = 00001000;
CAlarm_Mask_EveryMinute: Byte = 00001100;
CAlarm_Mask_Every10Minutes: Byte = 00010000;
CAlarm_Mask_EveryHour: Byte = 00010100;
CAlarm_Mask_OnceADay: Byte = 00011000;
CAlarm_Mask_OnceAWeek: Byte = 00011100;
```

```

CAlarm_Mask_OnceAMonth: Byte      = %00100000;
CAlarm_Mask_OnceAYear:  Byte      = %00100100;

```

CAlarm_EnableChime – if set, makes the alarm trigger indefinitely. If cleared, the alarm will trigger the number of times set by [SetRTCCAlarmRepeat](#) routine.

CAlarm_Mask_EveryHalfSecond – if chosen, makes the alarm trigger every half of a second.

CAlarm_Mask_EverySecond – if chosen, makes the alarm trigger every second.

CAlarm_Mask_Every10Seconds – if chosen, makes the alarm trigger every 10 seconds.

CAlarm_Mask_EveryMinute – if chosen, makes the alarm trigger every minute.

CAlarm_Mask_Every10Minutes – if chosen, makes the alarm trigger every 10 minutes.

CAlarm_Mask_EveryHour – if chosen, makes the alarm trigger every hour.

CAlarm_Mask_OnceADay – if chosen, makes the alarm trigger once a day.

CAlarm_Mask_OnceAWeek – if chosen, makes the alarm trigger once a week.

CAlarm_Mask_OnceAMonth – if chosen, makes the alarm trigger once a month.

CAlarm_Mask_OnceAYear – if chosen, makes the alarm trigger once a year.

3.3. Day of week constants

The day of week constants are internally used by the [GetRTCCDateTimeStr](#) routine. They are declared in the interface section to make them available to user.

const

```

CRTCC_Sunday: Byte = 0;
CRTCC_Monday: Byte = 1;
CRTCC_Tuesday: Byte = 2;
CRTCC_Wednesday: Byte = 3;
CRTCC_Thursday: Byte = 4;
CRTCC_Friday: Byte = 5;
CRTCC_Saturday: Byte = 6;

```

3.4. DeepSleep GP Persistent selector constants

These constants are used by the RTCC state backup routines.

const

```

CDSGPRSel_R0: Byte = 0;
CDSGPRSel_R1: Byte = 1;

```

CDSGPRSel_R0 – if chosen, the **DSGPR0** register is used.

CDSGPRSel_R1 – if chosen, the **DSGPR1** register is used.

3.5. GetRTCCDateTimeStr constants

The [GetRTCCDateTimeStr](#) routine can return the current RTCC Date/Time, alarm Date/Time and elapsed time to next alarm. To select one of these, there is used one of the following constants:

const

```

CRTCC_GetStrRTCC: Byte = 0;
CRTCC_GetStrAlarm: Byte = 1;
CRTCC_GetStrElapsedToNextAlarm: Byte = 2;

```

CRTCC_GetStrRTCC – if chosen, [GetRTCCDateTimeStr](#) routine returns RTCC Date/Time.

CRTCC_GetStrAlarm – if chosen, [GetRTCCDateTimeStr](#) routine returns alarm Date/Time.

CRTCC_GetStrElapsedToNextAlarm – if chosen, [GetRTCCDateTimeStr](#) routine returns elapsed time to next alarm.

3.6. DateTime format constants

When choosing RTCC Date/Time or alarm Date/Time for [GetRTCCDateTimeStr](#) routine, the year / month / date format can be selected from one of these constants:

const

```
CDateFmt_YYMMDD: Byte = 0;  
CDateFmt_YYYYMMDD: Byte = 1;  
CDateFmt_MMDDYY: Byte = 2;  
CDateFmt_MMDDYYYY: Byte = 3;  
CDateFmt_DDMMYY: Byte = 4;  
CDateFmt_DDMMYYYY: Byte = 5;
```

CDateFmt_YYMMDD – if chosen, the returned string is formatted as: YY.MM.DD HH:MN:SS

CDateFmt_YYYYMMDD – if chosen, the returned string is formatted as: YYYY.MM.DD HH:MN:SS

CDateFmt_MMDDYY – if chosen, the returned string is formatted as: MM.DD.YY HH:MN:SS

CDateFmt_MMDDYYYY – if chosen, the returned string is formatted as: MM.DD.YYYY HH:MN:SS

CDateFmt_DDMMYY – if chosen, the returned string is formatted as: DD.MM.YY HH:MN:SS

CDateFmt_DDMMYYYY – if chosen, the returned string is formatted as: DD.MM.YYYY HH:MN:SS

The *DateTime format constants* do not affect the returned string of [GetRTCCDateTimeStr](#) routine when `CRTCC_GetStrElapsedToNextAlarm` is chosen.

3.7. Config backup mask constant

This constant is used to mask the backup variable kept in the DeepSleep persistent register. Its purpose is to avoid writing to bits that are not used for this backup purpose. Thus the user can set these remaining bits according to other data that must be backed up. The current version of the library uses 6 bits that must be backed up.

const

```
CRTCC_ConfiguredAfterPowerUp_Mask: Byte = %00111111;
```


4. Global variables

The library makes available to the user only the "configured state" of the RTCC and alarm in a single byte variable. This variable is saved and restored from the selected DeepSleep persistent register using the *RTCC state backup routines*.

var

```
RTCCConfiguredAfterPowerUp: Byte;
```

The firmware must be written in such a way that for the first use of this variable, it has to be reset to 0 at the beginning of the program. Once the microcontroller enters deep sleep mode and then wakes up, this variable should not be reset anymore, it should be used only with the restore routine. To check whether the microcontroller wakes up or just powers up the user should check the DS bit from the WDTCON register.

The bits of *RTCCConfiguredAfterPowerUp* variable are defined as:

```
bit_0 = 1 - set after calling EnableRTCCModule  
bit_1 = 1 - set after calling SetRTCCTimeBCD, SetRTCCTimeDec or SetRTCCDateTime BCD/Dec  
bit_2 = 1 - set after calling SetRTCCDateBCD, SetRTCCDateDec or SetRTCCDateTime BCD/Dec  
bit_3 = 1 - set after calling SetRTCCAlarmBCD or SetRTCCAlarmDec  
bit_4 = 1 - set after calling SetRTCCCalibrationValue  
bit_5 = 1 - set after calling EnableRTCCAlarm
```

5. Usage example

This example sets date/time for RTCC and clock as constants. They should be set from other sources, e.g. a time server. However, the routines do not count for delays.

```
begin
  Delay_ms(100);
  InitPIC;      // do your init here (without RTCC init)

  //checking the wakeup source might need testing more other bits than WDTCON.DS - this is only an example.
  //the DeepSleep BOR makes the persistent registers lose their date, so other bits should be checked
  if WDTCON.DS = 0 then //reset from other source than deep sleep
  begin
    //power up, no RTCC configuration, so configure something
    RTCCConfiguredAfterPowerUp := 0; //clear the settings for the first use
    SetRTCCCalibrationValue(-10); //My RTCC runs faster than normal, so make it run slower !

    EnableRTCCModule(CRTCC_StartTlOsc or CRTCC_DisIntOnReadWrite or CRTCC_RTCCOutputEnable);
    SetRTCCDateTImeDec(11, 1, 8, CRTCC_Monday, 15, 17, 20); // set Monday, 2011, January 8, 15:17:20
    SetRTCCDateStrFormat(CDateFmt_YYYYMMDD);

    SetRTCCAlarmDec(CAlarm_Mask_OnceAYear, 1, 8, CRTCC_Monday, 15, 18, 20); //set Monday, January 8, 15:1
8:20
    SetRTCCAlarmRepeat(50); //alarm will trigger 50 times after RTCC/alarm counters match
    EnableRTCCAlarm(CAlarm_Mask_Every10Minutes); //change alarm mask if want to trigger earlier
  end
  else //reset from deep sleep wake up - the RTCC is configured already, just restore the setting
    RestoreRTCCConfigStateFromDeepSleepPersistentRegs(CDSGPRSel_R0);

    //.....
    //somewhere in code, before entering deep sleep mode, save the RTCC "config state"
    SaveRTCCConfigStateToDeepSleepPersistentRegs(CDSGPRSel_R0);
    //.....
end.
```